



P01-02 · T2–T3 aura-core — Error Taxonomy, Typed IDs, Paths, Clock & Consent



These four files are copied **verbatim**. They are the vocabulary every other crate speaks. After they land, run `cargo xtask contracts` and they are frozen: changing a field later needs an ADR with a grep blast radius.

1. `crates/aura-core/src/contract/error.rs` — FROZEN

```
///! FROZEN CONTRACT. Changes require an ADR per 13-CONTRACT-FREEZE-AND-ADR.md.  
///!  
///! Every fallible operation in AURA returns Result<T, AuraError>. There is no  
///! other error type crossing a module boundary. A caller can always answer  
///! three questions from the error alone: how bad is it, what should I do, and  
///! what do I tell the photographer.  
  
use std::fmt;  
  
use serde::{Deserialize, Serialize};  
  
/// How bad is it.
```

```

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord,
Serialize, Deserialize)]
#[serde(rename_all = "snake_case")]
pub enum Severity {
    /// Informational. The run continues and quality is unaffected.
    Warning,
    /// The run continues with reduced capability. Must be surfaced in the report.
    Degraded,
    /// One photo could not be processed. The other 3,999 continue.
    ItemFailed,
    /// This run cannot continue, but the catalog is intact and resumable.
    RunBlocking,
    /// The application cannot continue safely. Data integrity may be at risk.
    Fatal,
}

/// What the caller should do about it.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
#[serde(rename_all = "snake_case")]
pub enum Recovery {
    /// Transient. Retry with backoff, bounded attempts.
    Retry,
    /// Use a lower-quality or slower path that is known to work.
    Fallback,
    /// Set this item aside with its reason and keep going.
    Quarantine,
    /// Stop the run cleanly. Persist state so resume is possible.
    Halt,
}

```

```

    /// Only a human can decide. Present the choice, never guess.
    AskUser,
}

/// AURA-<DOMAIN>-<NNNN>. Ranges are allocated in 11-ERROR-TAXONOMY.md:
/// IO 1000, RAW 2000, DB 3000, GPU 4000, ML 5000, CLOUD 6000, JOB 7000,
/// EXPORT 8000, SEC 9000.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Serialize, Deserialize)]
pub struct ErrorCode(pub &'static str);

impl fmt::Display for ErrorCode {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        f.write_str(self.0)
    }
}

impl ErrorCode {
    /// Domain segment, e.g. "IO".
    #[must_use]
    pub fn domain(&self) -> &'static str {
        self.0.split('-').nth(1).unwrap_or("UNKNOWN")
    }

    /// Public runbook URL shown in the UI and in support replies.
    #[must_use]
    pub fn runbook_url(&self) -> String {
        format!("https://aura.app/e/{}", self.0)
    }
}

```

```

/// The one error type. Construct it with the helpers, never
with struct
/// literal syntax outside this crate.
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct AuraError {
    pub code: ErrorCode,
    pub severity: Severity,
    pub recovery: Recovery,
    /// Engineer-facing. May contain paths and technical details. Never shown raw.
    pub detail: String,
    /// Photographer-facing. One sentence, no jargon, says what to do next.
    pub user_message: String,
    /// Structured context for logs. Keys are stable; values are redacted
    /// according to the data-class rules in 24-PRIVACY-CONSENT-AND-LEGAL.md.
    pub context: Vec<(String, String)>,
    /// Chained cause, formatted. We keep a string rather than a boxed error so
    /// the type stays Clone, Send, Sync and serialisable into the event stream.
    pub cause: Option<String>,
}

impl fmt::Display for AuraError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "[{}] {}", self.code, self.detail)?;
        if let Some(cause) = &self.cause {
            write!(f, " (cause: {cause})")?;
        }
        Ok(())
    }
}

```

```

impl std::error::Error for AuraError {}

impl AuraError {
    #[must_use]
    pub fn new(
        code: ErrorCode,
        severity: Severity,
        recovery: Recovery,
        detail: impl Into<String>,
        user_message: impl Into<String>,
    ) -> Self {
        Self {
            code,
            severity,
            recovery,
            detail: detail.into(),
            user_message: user_message.into(),
            context: Vec::new(),
            cause: None,
        }
    }

    #[must_use]
    pub fn with_context(mut self, key: impl Into<String>, value: impl Into<String>) -> Self {
        self.context.push((key.into(), value.into()));
        self
    }

    #[must_use]
    pub fn with_cause(mut self, cause: &dyn std::error::Error) -> Self {
        self.cause = Some(cause.to_string());
        self
    }
}

```

```

    /// True when the caller may retry without human input.
    #[must_use]
    pub fn is_retryable(&self) -> bool {
        matches!(self.recovery, Recovery::Retry)
    }

    /// True when the run must stop.
    #[must_use]
    pub fn stops_run(&self) -> bool {
        matches!(self.severity, Severity::RunBlocking | Severity::Fatal)
    }
}

pub type AuraResult<T> = Result<T, AuraError>;

```

Why `cause` is a `String`

A boxed `dyn Error` cannot be `Clone` or `Serialize`. Every error in AURA has to travel three places: into the SQLite `task` row for resume, into the JSON event stream for the UI, and into the diagnostics bundle. Formatting at construction time costs a few microseconds on a path that has already failed and buys us a type that works everywhere.

2. `crates/aura-core/errors.toml` — the registry

One entry per code. CI fails if a code appears in Rust source but not here, or if an entry lacks a runbook file.

```

[[error]]
code          = "AURA-IO-1001"
severity      = "run_blocking"
recovery      = "ask_user"
summary       = "Source folder does not exist"
user_message  = "That folder is not there any more. Reconnect

```

the drive or choose the folder again."

runbook = "docs/runbooks/AURA-I0-1001.md"

introduced = "PHASE-01"

[[error]]

code = "AURA-I0-1002"

severity = "run_blocking"

recovery = "ask_user"

summary = "Permission denied reading the source folder"

user_message = "AURA is not allowed to read that folder. Grant access in system settings, then try again."

runbook = "docs/runbooks/AURA-I0-1002.md"

introduced = "PHASE-01"

[[error]]

code = "AURA-I0-1003"

severity = "run_blocking"

recovery = "retry"

summary = "Volume disconnected during a run"

user_message = "The drive was disconnected. Plug it back in and AURA will carry on from where it stopped."

runbook = "docs/runbooks/AURA-I0-1003.md"

introduced = "PHASE-01"

[[error]]

code = "AURA-I0-1004"

severity = "run_blocking"

recovery = "halt"

summary = "Disk full"

user_message = "The disk is full. Free up space and AURA will resume; nothing done so far is lost."

runbook = "docs/runbooks/AURA-I0-1004.md"

introduced = "PHASE-01"

[[error]]

code = "AURA-I0-1005"

```
severity      = "item_failed"
recovery      = "quarantine"
summary       = "Path exceeds the operating system limit"
user_message  = "One file is nested too deeply for Windows to
open. It has been set aside; the rest imported normally."
runbook       = "docs/runbooks/AURA-I0-1005.md"
introduced    = "PHASE-01"
```

```
[[error]]
code          = "AURA-I0-1006"
severity      = "item_failed"
recovery      = "quarantine"
summary       = "File unreadable or truncated"
user_message  = "One file could not be read and was set aside.
Check the card in your camera before formatting it."
runbook       = "docs/runbooks/AURA-I0-1006.md"
introduced    = "PHASE-01"
```

```
[[error]]
code          = "AURA-I0-1007"
severity      = "item_failed"
recovery      = "retry"
summary       = "File changed while being read"
user_message  = "A file was still being written while AURA rea
d it. It will be retried."
runbook       = "docs/runbooks/AURA-I0-1007.md"
introduced    = "PHASE-01"
```

```
[[error]]
code          = "AURA-I0-1008"
severity      = "run_blocking"
recovery      = "ask_user"
summary       = "Catalog location is inside a cloud-synced fol
der"
user_message  = "Catalogs cannot live in Dropbox, iCloud Driv
e, OneDrive or Google Drive; sync corrupts them. Choose a loc
```

```

al folder."
runbook      = "docs/runbooks/AURA-I0-1008.md"
introduced   = "PHASE-01"

[[error]]
code         = "AURA-I0-1010"
severity     = "item_failed"
recovery     = "quarantine"
summary      = "Zero-byte file"
user_message = "One file is empty, which usually means a failed card transfer. It has been set aside."
runbook      = "docs/runbooks/AURA-I0-1010.md"
introduced   = "PHASE-01"

[[error]]
code         = "AURA-DB-3005"
severity     = "run_blocking"
recovery     = "ask_user"
summary      = "Another AURA instance holds the catalog write lock"
user_message = "This catalog is already open in another AURA window. Close it and try again."
runbook      = "docs/runbooks/AURA-DB-3005.md"
introduced   = "PHASE-01"

```

The remaining DB and JOB codes follow the identical shape and are listed in the hub table; the runbook bodies are in **P01-09**.

The registry test

```

// crates/aura-core/tests/error_registry.rs
use std::{collections::BTreeSet, fs, path::Path};

#[derive(serde::Deserialize)]
struct Registry { error: Vec<Entry> }

```

```

#[derive(serde::Deserialize)]
struct Entry {
    code: String, severity: String, recovery: String,
    summary: String, user_message: String, runbook: String, i
ntroduced: String,
}

#[test]
fn every_registered_code_is_well_formed_and_has_a_runbook() {
    let text = fs::read_to_string("errors.toml").expect("erro
rs.toml");
    let reg: Registry = toml::from_str(&text).expect("parse e
rrors.toml");
    let mut seen = BTreeSet::new();

    for e in &reg.error {
        assert!(seen.insert(e.code.clone()), "duplicate code
{}", e.code);

        let parts: Vec<&str> = e.code.split('-').collect();
        assert_eq!(parts.len(), 3, "malformed code {}", e.cod
e);

        assert_eq!(parts[0], "AURA");
        let n: u32 = parts[2].parse().expect("numeric suffi
x");

        let expected_range = match parts[1] {
            "IO" => 1000..2000, "RAW" => 2000..3000, "DB" =>
3000..4000,
            "GPU" => 4000..5000, "ML" => 5000..6000, "CLOUD"
=> 6000..7000,
            "JOB" => 7000..8000, "EXPORT" => 8000..9000, "SE
C" => 9000..10000,
            other => panic!("unknown domain {other}"),
        };
        assert!(expected_range.contains(&n), "{} is outside i
ts domain range", e.code);
    }
}

```

```

        // A user message that names a file path or an exception
        // leaks detail.
        assert!(!e.user_message.is_empty());
        assert!(e.user_message.len() <= 180, "{} user_message
too long", e.code);
        for banned in ["Error:", "errno", "panicked", "None",
"unwrap", "\\\"", ":","] {
            assert!(!e.user_message.contains(banned),
                "{} leaks technical detail: {banned}", e.code);
        }

        assert!(Path::new(&e.runbook).exists(), "{} has no run
book at {}", e.code, e.runbook);
        assert!(!e.summary.is_empty() && !e.introduced.is_empt
y());
        assert!(["warning", "degraded", "item_failed", "run_bloc
king", "fatal"].contains(&e.severity.as_str()));
        assert!(["retry", "fallback", "quarantine", "halt", "ask_
user"].contains(&e.recovery.as_str()));
    }
}

#[test]
fn every_code_used_in_source_is_registered() {
    let text = fs::read_to_string("errors.toml").expect("erro
rs.toml");
    let reg: Registry = toml::from_str(&text).expect("pars
e");
    let registered: BTreeSet<_> = reg.error.iter().map(|e| e.
code.clone()).collect();

    let mut used = BTreeSet::new();
    for entry in walkdir::WalkDir::new("../..").max_depth(6).
into_iter().filter_map(Result::ok) {

```

```

        let p = entry.path();
        if p.extension().is_some_and(|e| e == "rs") && !p.to_string_lossy().contains("target") {
            let src = fs::read_to_string(p).unwrap_or_default();
            for cap in src.match_indices("AURA-") {
                let tail = &src[cap.0..(cap.0 + 13).min(src.len())];
                if tail.len() == 13 { used.insert(tail.to_string()); }
            }
        }
        let missing: Vec<_> = used.difference(&registered).collect();
        assert!(missing.is_empty(), "codes used but not registered: {missing:?}");
    }
}

```

The `user_message` banned-substring assertion is the small test that prevents the most common support failure: a dialog reading `Error: 0s { code: 2, kind: NotFound }`, which teaches the photographer nothing and generates a ticket.

3. Constructor helpers — `crates/aura-core/src/errors/io.rs`

Callers never assemble severity and recovery by hand; they call a named constructor. That is how the registry and the code stay in sync.

```

use std::path::Path;

use crate::contract::error::{AuraError, ErrorCode, Recovery, Severity};
use crate::redact::redact_path;

pub const IO_NOT_FOUND: ErrorCode = ErrorCode("AURA-IO-1

```

```

001");
pub const IO_PERMISSION: ErrorCode = ErrorCode("AURA-IO-1
002");
pub const IO_DISCONNECTED: ErrorCode = ErrorCode("AURA-IO-1
003");
pub const IO_DISK_FULL: ErrorCode = ErrorCode("AURA-IO-1
004");
pub const IO_PATH_TOO_LONG: ErrorCode = ErrorCode("AURA-IO-1
005");
pub const IO_UNREADABLE: ErrorCode = ErrorCode("AURA-IO-1
006");
pub const IO_CHANGED: ErrorCode = ErrorCode("AURA-IO-1
007");
pub const IO_CLOUD_SYNCED: ErrorCode = ErrorCode("AURA-IO-1
008");
pub const IO_ZERO_BYTE: ErrorCode = ErrorCode("AURA-IO-1
010");

#[must_use]
pub fn not_found(path: &Path) -> AuraError {
    AuraError::new(
        IO_NOT_FOUND, Severity::RunBlocking, Recovery::AskUse
r,
        format!("path does not exist: {}", path.display()),
        "That folder is not there any more. Reconnect the dri
ve or choose the folder again.",
    ).with_context("path", redact_path(path))
}

#[must_use]
pub fn unreadable(path: &Path, cause: &dyn std::error::Error)
-> AuraError {
    AuraError::new(
        IO_UNREADABLE, Severity::ItemFailed, Recovery::Quaran
tine,
        format!("unreadable file: {}", path.display()),

```

```

        "One file could not be read and was set aside. Check
the card before formatting it.",
    ).with_context("path", redact_path(path)).with_cause(cause)
}

/// Map a std::io::Error to the right code. This is the only
place in the
/// codebase permitted to inspect io::ErrorKind.
#[must_use]
pub fn from_io(err: &std::io::Error, path: &Path) -> AuraError {
    use std::io::ErrorKind as K;
    match err.kind() {
        K::NotFound => not_found(path),
        K::PermissionDenied => AuraError::new(
            IO_PERMISSION, Severity::RunBlocking, Recovery::AskUser,
            format!("permission denied: {}", path.display()),
            "AURA is not allowed to read that folder. Grant access in system settings, then try again.",
        ).with_context("path", redact_path(path)),
        K::StorageFull => AuraError::new(
            IO_DISK_FULL, Severity::RunBlocking, Recovery::Halt,
            "disk full",
            "The disk is full. Free up space and AURA will resume; nothing done so far is lost.",
        ),
        K::InvalidData | K::UnexpectedEof => unreadable(path, err),
        _ => unreadable(path, err),
    }
}

```

4. Redaction — `crates/aura-core/src/redact.rs`

Filenames at a wedding are class C2: `Sarah-and-Tom-Wedding/getting-ready/IMG_0421.CR2` identifies real people. Logs are class C1. Therefore paths are redacted before they enter a log line, and the full path lives only in the local catalog.

```
use std::path::Path;

/// Keep the shape of a path for debugging, remove the identifying content.
/// `/Users/nina/Weddings/Sarah-and-Tom/raw/IMG_0421.CR2`
/// becomes `/2-deep/IMG_0421.CR2` -> `/**/*.CR2`
#[must_use]
pub fn redact_path(path: &Path) -> String {
    let ext = path.extension().and_then(|e| e.to_str()).unwrap_or("none");
    let depth = path.components().count();
    format!("
```

5. `crates/aura-core/src/contract/ids.rs` — FROZEN

```
//! FROZEN CONTRACT. Typed IDs. A ProjectId can never be passed where a PhotoId
```

//! is expected, which eliminates an entire class of catalog-corrupting bug.

```
use std::fmt;
```

```
use serde::{Deserialize, Serialize};
```

```
use uuid::Uuid;
```

```
macro_rules! typed_id {
```

```
    ($name:ident, $prefix:literal) => {
```

```
        #[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Serialize, Deserialize)]
```

```
        #[serde(transparent)]
```

```
        pub struct $name(Uuid);
```

```
        impl $name {
```

```
            /// New random v7 ID. Time-ordered, so index locality is good and
```

```
            /// insertion order is stable within a run.
```

```
            #[must_use]
```

```
            pub fn new() -> Self { Self(Uuid::now_v7()) }
```

```
            #[must_use]
```

```
            pub const fn from_uuid(u: Uuid) -> Self { Self(u)
```

```
        }
```

```
            #[must_use]
```

```
            pub const fn as_uuid(&self) -> &Uuid { &self.0 }
```

```
            /// Canonical text form stored in SQLite: prefixed, lower case.
```

```
            #[must_use]
```

```
            pub fn to_db(&self) -> String { format!("{_}_{}", $prefix, self.0.as_hyphenated()) }
```

```
            pub fn from_db(s: &str) -> Result<Self, IdParseError>
```

```

ror> {
    let rest = s.strip_prefix(concat!($prefix,
    "_"))
    .ok_or(IdParseError::WrongPrefix { expected: $prefix })?;
    Uid::parse_str(rest).map(Self).map_err(|_| IdParseError::NotAUuid)
}

impl Default for $name { fn default() -> Self { Self::new() } }

impl fmt::Display for $name {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        f.write_str(&self.to_db())
    }
}

};
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, thiserror::Error)]
pub enum IdParseError {
    #[error("expected prefix {expected}_")]
    WrongPrefix { expected: &'static str },
    #[error("not a valid uuid")]
    NotAUuid,
}

typed_id!(ProjectId, "prj");
typed_id!(PhotoId, "pht");
typed_id!(FileId, "fil");
typed_id!(RunId, "run");
typed_id!(ImportId, "imp");

```

```

/// Content address: BLAKE3 of the file bytes. Two files with
the same digest
/// are the same file, no matter what they are called or wher
e they live.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord,
Hash, Serialize, Deserialize)]
#[serde(transparent)]
pub struct ContentHash([u8; 32]);

impl ContentHash {
    #[must_use]
    pub const fn from_bytes(b: [u8; 32]) -> Self { Self(b) }

    #[must_use]
    pub fn to_hex(&self) -> String {
        let mut s = String::with_capacity(64);
        for byte in &self.0 {
            s.push_str(&format!("{byte:02x}"));
        }
        s
    }

    pub fn from_hex(hex: &str) -> Result<Self, IdParseError>
    {
        if hex.len() != 64 { return Err(IdParseError::NotAUUID); }

        let mut out = [0u8; 32];
        for (i, slot) in out.iter_mut().enumerate() {
            let pair = hex.get(i * 2..i * 2 + 2).ok_or(IdParseError::NotAUUID)?;
            *slot = u8::from_str_radix(pair, 16).map_err(|_| IdParseError::NotAUUID)?;
        }
        Ok(Self(out))
    }
}

```

```

}

impl fmt::Display for ContentHash {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result
    { f.write_str(&self.to_hex()) }
}

```

Why UUID v7 and not v4

V7 embeds a millisecond timestamp in the high bits, so IDs sort chronologically. In SQLite that keeps B-tree inserts appending at the right edge of the index instead of scattering across pages. On a 200,000-row catalog, the difference in import time is measurable, and it costs nothing.

Round-trip property test

```

// crates/aura-core/tests/ids.rs
use aura_core::contract::ids::{ContentHash, PhotoId, ProjectId};
use proptest::prelude::*;

#[test]
fn db_round_trip() {
    let id = PhotoId::new();
    assert_eq!(PhotoId::from_db(&id.to_db()).expect("parse"),
id);
}

#[test]
fn prefixes_are_not_interchangeable() {
    let project = ProjectId::new();
    assert!(PhotoId::from_db(&project.to_db()).is_err(),
"a project id must never parse as a photo id");
}

proptest! {

```

```

#[test]
fn hash_hex_round_trip(bytes in proptest::array::uniform3
2(any::<u8>())) {
    let h = ContentHash::from_bytes(bytes);
    prop_assert_eq!(ContentHash::from_hex(&h.to_hex()).expect("parse"), h);
}
}

```

6. crates/aura-core/src/clock.rs

`SystemTime::now` is banned by Clippy. All time reads go through here, so tests can freeze time and the chaos suite can jump it backwards.

```

use std::sync::Arc;

use time::OffsetDateTime;

pub trait Clock: Send + Sync + std::fmt::Debug {
    fn now_utc(&self) -> OffsetDateTime;
    /// Monotonic milliseconds since process start. Never goes
    s backwards, so
    /// lease expiry and progress ETAs are safe against NTP c
    orrections.
    fn monotonic_ms(&self) -> u64;
}

#[derive(Debug)]
pub struct SystemClock {
    start: std::time::Instant, // DETERMINISM: monotonic bas
    e, never serialised
}

impl Default for SystemClock {
    fn default() -> Self { Self { start: std::time::Instant::

```

```

now() } }
}

impl Clock for SystemClock {
    fn now_utc(&self) -> OffsetDateTime {
        OffsetDateTime::now_utc() // DETERMINISM: single permitted wall-clock read
    }
    fn monotonic_ms(&self) -> u64 {
        u64::try_from(self.start.elapsed().as_millis()).unwrap_or(u64::MAX)
    }
}

/// Test and chaos clock. Time only moves when you move it.
#[derive(Debug, Default)]
pub struct FixedClock {
    utc: parking_lot::Mutex<Option<OffsetDateTime>>,
    mono: std::sync::atomic::AtomicU64,
}

impl FixedClock {
    #[must_use]
    pub fn at(utc: OffsetDateTime) -> Arc<Self> {
        Arc::new(Self { utc: parking_lot::Mutex::new(Some(utc)), mono: 0.into() })
    }
    pub fn advance_ms(&self, ms: u64) {
        self.mono.fetch_add(ms, std::sync::atomic::Ordering::SeqCst);
        let mut guard = self.utc.lock();
        if let Some(t) = guard.as_mut() {
            *t += time::Duration::milliseconds(i64::try_from(ms).unwrap_or(0));
        }
    }
}

```

```

    /// Simulate an NTP correction or a user changing the sys
    tem clock.
    pub fn set_wall_clock(&self, utc: OffsetDateTime) { *sel
f.utc.lock() = Some(utc); }
}

impl Clock for FixedClock {
    fn now_utc(&self) -> OffsetDateTime {
        self.utc.lock().unwrap_or(OffsetDateTime::UNIX_EPOCH)
    }
    fn monotonic_ms(&self) -> u64 { self.mono.load(std::syn
c::atomic::Ordering::SeqCst) }
}

```

The monotonic-versus-wall-clock split matters concretely: a lease that expires based on wall-clock time will expire instantly when the photographer's laptop syncs its clock after a flight, and the job graph will reclaim thousands of healthy running tasks at once.

7. `crates/aura-core/src/paths.rs`

```

use std::path::{Path, PathBuf};

use crate::contract::error::{AuraError, AuraResult, Recovery,
Severity};
use crate::errors::io::IO_CLOUD_SYNCED;

/// Every location AURA writes to, resolved once at startup.
#[derive(Debug, Clone)]
pub struct AppPaths {
    /// Catalogs, previews, models. Large, local, never synce
    d.
    pub data_dir: PathBuf,
    /// Settings and keychain references. Small.
    pub config_dir: PathBuf,
}

```

```

    /// Rotating logs and crash reports.
    pub log_dir: PathBuf,
    /// Preview pyramid cache. Deletable at any time without
    data loss.
    pub cache_dir: PathBuf,
}

impl AppPaths {
    pub fn resolve() -> AuraResult<Self> {
        let data = dirs::data_dir().ok_or_else(|| AuraError::
new(
            crate::errors::io::IO_NOT_FOUND, Severity::Fatal,
Recovery::AskUser,
            "no platform data directory",
            "AURA could not find a place to store its files.
Contact support.",
        ))?.join("AURA");
        let config = dirs::config_dir().unwrap_or_else(|| dat
a.clone()).join("AURA");
        Ok(Self {
            log_dir: data.join("logs"),
            cache_dir: dirs::cache_dir().unwrap_or_else(|| da
ta.clone()).join("AURA"),
            data_dir: data,
            config_dir: config,
        })
    }
}

/// Folder-name markers that indicate a cloud-sync engine own
s this directory.
const CLOUD_MARKERS: [&str] = &[
    "dropbox", "onedrive", "google drive", "googledrive", "ic
loud",
    "com~apple~clouddocs", "pcloud", "box sync", "mega", "yan
dexdisk",

```

```

    "nextcloud", "owncloud", "sync.com", "idrive", "tresori
t",
];

/// Refuse to put a SQLite catalog anywhere a sync client wil
l touch it.
/// A sync engine copying a WAL file mid-write produces a cor
rupt catalog and a
/// photographer who has lost a wedding. This is a hard refus
al, not a warning.
pub fn assert_not_cloud_synced(path: &Path) -> AuraResult<()>
{
    let lowered = path.to_string_lossy().to_lowercase();
    for marker in CLOUD_MARKERS {
        if lowered.contains(marker) {
            return Err(AuraError::new(
                IO_CLOUD_SYNCED, Severity::RunBlocking, Recov
ery::AskUser,
                format!("catalog path matches cloud marker
'{marker}'"),
                "Catalogs cannot live in a synced folder such
as Dropbox, iCloud Drive, OneDrive or Google Drive, because s
yncing corrupts them. Choose a folder on this computer.",
                ).with_context("marker", *marker));
        }
    }
    // Marker files used by sync clients even when the folder
is renamed.
    for ancestor in path.ancestors().take(8) {
        for probe in [".dropbox", ".dropbox.cache", ".iclou
d", "desktop.ini.onedrive"] {
            if ancestor.join(probe).exists() {
                return Err(AuraError::new(
                    IO_CLOUD_SYNCED, Severity::RunBlocking, R
ecovery::AskUser,
                    format!("sync marker file {probe} found i

```

```

n an ancestor directory"),
    "That folder is managed by a file-sync ap
p, which corrupts catalogs. Choose a folder on this compute
r.",
    ).with_context("probe", probe));
    }
    }
    }
    Ok(())
}

/// Windows refuses paths over 260 chars unless the extended
prefix is used.
/// Applying it unconditionally on Windows removes an entire
failure class.
#[must_use]
pub fn platform_safe(path: &Path) -> PathBuf {
    #[cfg(windows)]
    {
        let s = path.to_string_lossy();
        if s.len() > 240 && !s.starts_with("\\\\?\\") {
            return PathBuf::from(format!("\\\\?\\{s}"));
        }
    }
    path.to_path_buf()
}

```

The cloud-sync refusal is one of the four refusals from `27-ONBOARDING-AND-FIRST-RUN.md`. It is worth being blunt about: every desktop photo tool that allowed catalogs in Dropbox has a support forum full of people who lost work. Refusing at creation time costs one dialog; allowing it costs a wedding.

8. `crates/aura-core/src/contract/consent.rs` — FROZEN

Created in phase 01 even though nothing egresses until PHASE-04, because the consent row must exist from the moment a project exists. Adding it later means projects with no consent record and a migration that has to guess.

```
///! FROZEN CONTRACT. Default is all-false. There is no code path that sets a
///! consent flag other than an explicit photographer action.

use serde::{Deserialize, Serialize};
use time::OffsetDateTime;

use crate::contract::ids::ProjectId;

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct ProjectConsent {
    pub project_id: ProjectId,
    /// Filenames, EXIF, counts may go to a cloud model.
    pub cloud_metadata: bool,
    /// Downscaled derived imagery may go to a cloud model.
    pub cloud_derived_imagery: bool,
    /// Full-resolution client imagery may leave the machine.
    Rarely granted.
    pub cloud_full_imagery: bool,
    /// Anonymous product telemetry.
    pub telemetry: bool,
    /// This project's decisions may improve shared models.
    pub improve_models: bool,
    pub granted_at: Option<OffsetDateTime>,
    pub granted_by: Option<String>,
    pub expires_at: Option<OffsetDateTime>,
    pub revoked_at: Option<OffsetDateTime>,
}

impl ProjectConsent {
    /// The only constructor. Everything denied.
    #[must_use]
```

```

pub fn denied(project_id: ProjectId) -> Self {
    Self {
        project_id,
        cloud_metadata: false,
        cloud_derived_imagery: false,
        cloud_full_imagery: false,
        telemetry: false,
        improve_models: false,
        granted_at: None,
        granted_by: None,
        expires_at: None,
        revoked_at: None,
    }
}

#[must_use]
pub fn is_active(&self, now: OffsetDateTime) -> bool {
    self.revoked_at.is_none() && self.expires_at.is_none_
or(|e| e > now)
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deseri
alize)]
pub enum DataClass {
    /// Public or non-sensitive product data.
    C0Public,
    /// Operational: counts, timings, error codes.
    C1Operational,
    /// Identifying: filenames, imagery, EXIF GPS.
    C2Identifying,
    /// Biometric: face embeddings, face crops.
    C3Biometric,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]

```

```
pub enum Permit { Allowed, Denied { reason: &'static str } }

/// Phase 01 registers the gate; PHASE-04 is the first caller.
pub trait ConsentGate: Send + Sync {
    fn may_egress(&self, project: ProjectId, class: DataClass) -> Permit;
}
```

`C3Biometric` never has an allowing branch anywhere in the codebase, in any phase. Face embeddings stay on the machine. That is a product non-negotiable, so the enum variant exists purely so that the gate can refuse it explicitly and a test can assert the refusal.

```
// crates/aura-core/tests/consent.rs
use aura_core::contract::consent::*;
use aura_core::contract::ids::ProjectId;

#[test]
fn default_consent_denies_everything() {
    let c = ProjectConsent::denied(ProjectId::new());
    assert!(!c.cloud_metadata && !c.cloud_derived_imagery &&
!c.cloud_full_imagery);
    assert!(!c.telemetry && !c.improve_models);
    assert!(c.granted_at.is_none());
}

#[test]
fn no_source_file_can_construct_a_granting_default() {
    // Guard against a future refactor adding Default with true fields.
    let src = std::fs::read_to_string("src/contract/consent.rs").expect("read");
    assert!(!src.contains("impl Default for ProjectConsent"),
        "ProjectConsent must not have a Default impl; use den
```

```
ied());  
}
```

9. `crates/aura-core/src/lib.rs`

```
#![forbid(unsafe_code)]  
#![deny(  
    clippy::unwrap_used, clippy::expect_used, clippy::panic,  
    clippy::todo,  
    clippy::unimplemented, clippy::indexing_slicing, clippy::  
float_cmp,  
    clippy::disallowed_methods, clippy::disallowed_types,  
    missing_debug_implementations, unreachable_pub, rust_2018  
_idioms  
)]  
#![warn(clippy::pedantic)]  
#![allow(clippy::module_name_repetitions)]  
  
pub mod clock;  
pub mod contract {  
    pub mod consent;  
    pub mod error;  
    pub mod ids;  
}  
pub mod errors {  
    pub mod db;  
    pub mod io;  
    pub mod job;  
}  
pub mod paths;  
pub mod redact;  
  
pub use contract::error::{AuraError, AuraResult, ErrorCode, R  
ecovery, Severity};
```

```
pub use contract::ids::{ContentHash, FileId, ImportId, PhotoId, ProjectId, RunId};
```

10. Acceptance criteria for T2 and T3

- ☐ `error.rs`, `ids.rs`, `consent.rs` exist verbatim and appear in `contracts.lock`
- ☐ `errors.toml` contains all 17 phase-01 codes; `cargo test -p aura-core` proves every one is well formed, in range, unique, and has a runbook file
- ☐ The `user_message` leak test passes; deliberately adding `Error: 0s { code: 2 }` to one message makes it fail, output pasted
- ☐ `every_code_used_in_source_is_registered` passes; adding an unregistered `AURA-IO-1099` to a source file makes it fail, output pasted
- ☐ `PhotoId::from_db(project_id.to_db())` returns `Err`, proven by test
- ☐ `ContentHash` hex round-trip property test passes with 1,000 cases
- ☐ `assert_not_cloud_synced` refuses all 15 markers and a renamed folder containing `.dropbox`; a plain local path passes
- ☐ `platform_safe` prepends the extended prefix above 240 chars on Windows and is a no-op elsewhere
- ☐ `FixedClock` can move the wall clock backwards while `monotonic_ms` still increases, proven by test
- ☐ `ProjectConsent::denied` is the only constructor; the no-`Default` guard test passes
- ☐ Clippy clean, `check-banned.sh` clean, zero `ALLOW-BANNED` markers in `aura-core`

11. Claude Code prompt for T2 and T3

Act as `TLC` (Technical Lead, Core). Implement T2 and T3 of PHASE-01 from this page. Copy `error.rs`, `ids.rs` and `consent.rs` **verbatim** — do not improve, rename, reorder or add fields. Then write `errors.toml` with all 17 phase-01 codes from the hub table, the four constructor modules, `redact.rs`, `clock.rs`, `paths.rs`, and every test on this page.

Hard constraints: `aura-core` depends on no other workspace crate; no `HashMap` or `HashSet`; the only `OffsetDateTime::now_utc` call in the crate is inside `SystemClock`; no `user_message` may contain a path, an errno, or a Rust type name.

Evidence I expect pasted back: `cargo test -p aura-core` full output; the deliberate-failure runs for the leak test and the unregistered-code test with the failure text, then the passing rerun; `cargo clippy -p aura-core --all-targets -- -D warnings`; and `cargo xtask contracts` showing the three frozen files added to `contracts.lock`. Stop after T3.